

CXLMEMURING: Revolutionizing Memory Access Through Compiler-Driven Heterogeneous Execution

Yiwei Yang, Yusheng Zheng, Yuyi Li, Jianchang Su, Wei Zhang, Andi Quinn

Abstract

The emergence of Compute Express Link (CXL) technology promises to address memory capacity constraints through disaggregated memory pools, yet existing compiler infrastructures fail to exploit the heterogeneous computation opportunities presented by modern CXL-attached memory systems. Current approaches treat CXL memory as a simple capacity extension, missing critical optimization opportunities for coordinated computation and memory management across heterogeneous processors. We present CXLMEMURING, a compiler-driven framework that leverages a multi-level intermediate representation architecture to enable efficient asynchronous memory access patterns for CXL-attached memory. CXLMEMURING introduces CIRA (CXL Intermediate Representation for Asynchrony), a novel middle-layer IR that bridges high-level abstractions from ClangIR, TOSA, and SCF dialects with low-level heterogeneous backend code generation for x86 computation cores and Vortex RISC-V SIMT management cores. Through systematic offloading of last-level cache (LLC) management operations to dedicated Vortex RISC-V SIMT cores while maintaining computation on x86 processors, CXLMEMURING achieves up to 1.2-1.5 $\times$ speedup for pointer-chasing workloads and 1.1 $\times$ improvement in throughput for tensor operations compared to state-of-the-art CXL memory systems. Our evaluation, conducted using a real device Vortex cores within a CXL workflow, demonstrates that compiler-orchestrated heterogeneous execution can effectively hide CXL memory latency.

1 Introduction

The proliferation of memory-intensive applications in machine learning and high-performance computing has fundamentally challenged traditional memory hierarchy designs [6, 9, 12]. This capacity crisis coincides with the persistent memory wall problem, where the gap between processor and memory performance continues to widen despite architectural innovations [30, 45]. Compute Express Link (CXL) emerges

as a promising solution by enabling cache-coherent access to disaggregated memory pools [10, 36, 43], yet current software stacks fail to fully exploit the heterogeneous computing opportunities inherent in CXL-attached memory architectures [28, 40].

The fundamental challenge in CXL memory utilization extends beyond simple expansion of the capacity. While CXL provides load/store semantics with cache coherence, the inherent latency differential between local DRAM (approximately 80ns) and CXL-attached memory (150–300ns) creates performance cliffs that traditional compiler optimizations cannot address [36, 43]. Existing approaches treat CXL memory uniformly, missing opportunities for intelligent workload partitioning across heterogeneous processing elements commonly found in modern CXL memory expanders [28, 35, 40]. These expanders increasingly incorporate dedicated management processors alongside memory arrays, creating a heterogeneous computing environment that remains unexploited by current compiler infrastructures [11, 32, 42].

This paper presents CXLMEMURING, a compiler-driven framework that fundamentally reimagines memory access optimization for CXL-attached systems through coordinated heterogeneous execution. Our key insight lies in recognizing that effective CXL memory utilization requires not just hiding *latency* but actively leveraging the *computational capabilities* distributed throughout the memory system [30]. CXLMEMURING introduces a multi-level compilation strategy that systematically transforms high-level program representations into coordinated execution plans across x86 computation cores and Vortex RISC-V SIMT management cores embedded within CXL memory expanders [11, 42].

The technical foundation of CXLMEMURING rests on three interconnected innovations. First, we develop CIRA (CXL Intermediate Representation for Asynchrony), a novel middle-layer IR that captures asynchronous memory access patterns while maintaining semantic information necessary for heterogeneous backend targeting. CIRA serves as the convergence point for multiple frontend representations including ClangIR [22] for general computation, TOSA [20] for ten-

sor operations, and SCF for structured control flow [23–25], unifying these diverse abstractions into a common representation suitable for CXL-specific optimizations. Second, we implement a sophisticated backend partitioning strategy that assigns computational kernels to x86 processors while offloading LLC management operations to Vortex RISC-V SIMT cores, [11] achieving efficient cache line installation without disrupting primary computation. Third, we introduce adaptive runtime profiling that guides dynamic recompilation based on observed memory access patterns, enabling continuous optimization as workload characteristics evolve [1, 41].

This paper validates the architectural contributions of CXLMEMURING through a high-fidelity hardware methodology. We employ an intel Altera CXL FPGA composed of the Vortex RISC-V RTL, coupled with a two-pass execution model that accurately models the interaction between host x86 execution and offloaded accelerator tasks. Our results demonstrate the practical viability of compiler-orchestrated heterogeneous execution for CXL memory systems. Through comprehensive evaluation across pointer-chasing microbenchmarks, graph analytics workloads, and machine learning inference tasks, we show that CXLMEMURING achieves substantial performance improvements over both baseline CXL systems and state-of-the-art software prefetching approaches [1, 8, 29, 41].

In summary, our key contributions are:

- **CIRA IR Design:** We introduce CIRA, a domain-specific intermediate representation that unifies diverse frontend dialects (ClangIR, TOSA, SCF) and provides primitives for asynchronous, heterogeneous memory management.
- **Heterogeneous Compilation Stack:** We present a complete compiler backend that partitions execution between x86 host cores (computation) and Vortex RISC-V SIMT cores (memory orchestration).
- **Adaptive Runtime System:** We design a feedback-driven runtime that utilizes hardware performance counters to dynamically adjust prefetching strategies and offload granularity.
- **Rigorous Evaluation:** We validate CXLMEMURING using real device across diverse workloads, demonstrating up to $1.5\times$ speedup for pointer-chasing tasks and significant reductions in cache pollution.

2 Background and Motivation

2.1 CXL Memory Architecture and Coherence Protocols

Compute Express Link represents a convergence of memory and interconnect technologies, providing cache-coherent

access to remote memory resources through three distinct protocols: CXL.io for device discovery and enumeration, CXL.cache for device-initiated coherent accesses, and CXL.mem for host-initiated memory accesses. [10, 36, 43] The CXL.mem protocol particularly enables fine-grained 64-byte cache line transfers between host processors and attached memory devices, maintaining coherence through a bias-based mechanism that tracks ownership between host and device. [10, 43] This granularity presents both opportunities and challenges for compiler optimization, as traditional block-based prefetching strategies developed for page-granular systems prove ineffective for CXL’s cache-line-oriented architecture [1, 8, 17, 29].

Modern CXL memory expanders integrate computational capabilities in conjunction with memory arrays, typically incorporating management processors to handle memory controller logic, error correction, telemetry, and device-local orchestration [32]. These processors, in our target design, are realized as Vortex RISC-V SIMT cores that provide efficient fine-grained parallelism and are well-suited for memory-centric tasks. [11, 42] This SIMT-style compute capacity near memory creates opportunities for offloading memory-intensive operations, yet current software stacks treat these expanders as passive memory devices, missing critical optimization opportunities. [30]

2.2 Limitations of Current Compiler Approaches

Contemporary compiler infrastructures for heterogeneous memory systems fall into two categories: transparent runtime systems that hide memory heterogeneity from applications, and explicit programming models that burden developers with manual memory management. Transparent approaches like Linux’s memory tiering subsystem and Intel’s Sub-NUMA Clustering treat CXL memory as slower DRAM, relying on page migration and caching to mitigate latency differences. [16, 28, 35, 40] These systems operate at page granularity (typically 4KB), creating significant overhead for fine-grained access patterns common in pointer-chasing workloads. [28, 40] Moreover, they cannot leverage the computational capabilities of memory-side processors, as the operating system views CXL expanders merely as memory capacity extensions. [32]

Explicit programming models such as OpenMP target directives and CUDA Unified Memory provide programmer control over data placement but require extensive code modifications and domain expertise. [14, 31, 33] These approaches fundamentally assume a host-centric computation model where all processing occurs on the primary CPU or GPU, with memory serving purely as storage. This assumption breaks down in CXL environments where memory expanders contain capable processors such as Vortex RISC-V SIMT cores that could participate in computation. [11] Furthermore, existing compiler

intermediate representations lack abstractions for expressing asynchronous memory operations with heterogeneous execution targets, preventing effective optimization across the diverse processing elements in modern CXL systems [22, 23].

2.3 Overhead Analysis of Uncoordinated Memory Access

To quantify the performance impact of uncoordinated CXL memory access, we conducted a detailed analysis of memory-intensive workloads on a system with local DRAM and CXL-attached memory. Our measurements reveal that naive CXL memory access incurs not just the expected latency penalty but also significant indirect costs through cache pollution and pipeline stalls, consistent with prior studies of dependent cache misses and pointer-chasing workloads [15, 17, 26]. When traversing a linked list stored in CXL memory, each pointer dereference causes the processor to stall for 150–300ns, during which the out-of-order execution window becomes saturated with dependent instructions that cannot proceed. This stall propagation effect amplifies the base latency penalty by 2–3 $\times$, as the processor cannot effectively overlap independent work with memory access latency. [15, 30]

The overhead becomes particularly acute in workloads with irregular access patterns. Graph algorithms like breadth-first search experience 70% of execution time stalled on memory accesses when operating on CXL-resident data structures [5, 38, 40], and exhibit similar memory bottlenecks on disaggregated or remote memory systems. [26] Machine learning inference workloads exhibit similar patterns when accessing embedding tables in CXL memory, with random lookups causing frequent pipeline stalls [18, 44, 46]. These measurements motivate our compiler-driven approach that explicitly coordinates memory access patterns across heterogeneous processors to hide latency through computation overlap and intelligent prefetching [1, 30, 41].

2.4 Opportunities for Compiler-Driven Optimization

The convergence of CXL memory technology with heterogeneous processing capabilities creates unprecedented opportunities for compiler optimization. By treating the memory system as a distributed computing platform rather than passive storage, compilers can orchestrate coordinated execution plans that leverage all available computational resources, aligning with broader trends in processing-in-memory and near-data computing [15, 30]. The key insight lies in recognizing that memory access patterns exhibit sufficient regularity to enable compile-time analysis and optimization, even for apparently irregular workloads like pointer chasing. [1, 17] Through profile-guided compilation and runtime adaptation, compilers can learn application-specific access patterns and

generate specialized code for efficient CXL memory utilization [1, 41].

The presence of Vortex RISC-V SIMT cores in CXL expanders enables a division of labor between computation and memory management. While x86 cores excel at general computation with sophisticated out-of-order execution and SIMD capabilities, Vortex SIMT cores prove more power- and area-efficient for regular, massively parallel memory management tasks like cache line prefetching and LLC population [11, 30]. This heterogeneity suggests a compilation strategy that partitions work based on processor strengths: x86 cores execute computational kernels while Vortex cores handle memory orchestration, with the compiler coordinating their interaction through carefully scheduled asynchronous operations [22, 23].

3 System Design

3.1 Architectural Overview

CXLMEMURING implements a vertically integrated compilation stack that transforms high-level program representations into coordinated execution plans for heterogeneous CXL memory systems. The architecture comprises three primary components operating in concert: a *multi-dialect frontend* that ingests programs through ClangIR, TOSA, and SCF representations [20, 22–25]; the CXLMEMURING *middle layer* that provides unified abstractions for asynchronous memory operations; a *dual-backend code generator* that targets x86 computation cores and Vortex RISC-V SIMT management cores. [11] This design philosophy departs from traditional monolithic compilers by explicitly acknowledging and exploiting the heterogeneous nature of modern memory systems. [30]

The compilation flow begins with source programs in standard languages like C++ or domain-specific frameworks for machine learning. These programs undergo initial transformation into one of three frontend representations based on their characteristics: ClangIR for general imperative code preserving C++ semantics, TOSA for tensor operations common in machine learning workloads, and SCF for structured control flow patterns amenable to parallelization [20, 22, 24, 25]. This multi-dialect approach ensures that domain-specific optimizations can be applied early while maintaining a path to unified optimization in subsequent stages. The frontend representations converge at the CIRA layer, which provides a common abstraction for memory operations independent of their original expression. [23]

The CIRA intermediate representation serves as the critical abstraction layer that enables heterogeneous backend targeting. Unlike traditional IRs that assume a uniform memory model, CIRA explicitly represents asynchronous memory operations, memory hierarchy hints, and synchronization primitives necessary for coordinated execution across multiple

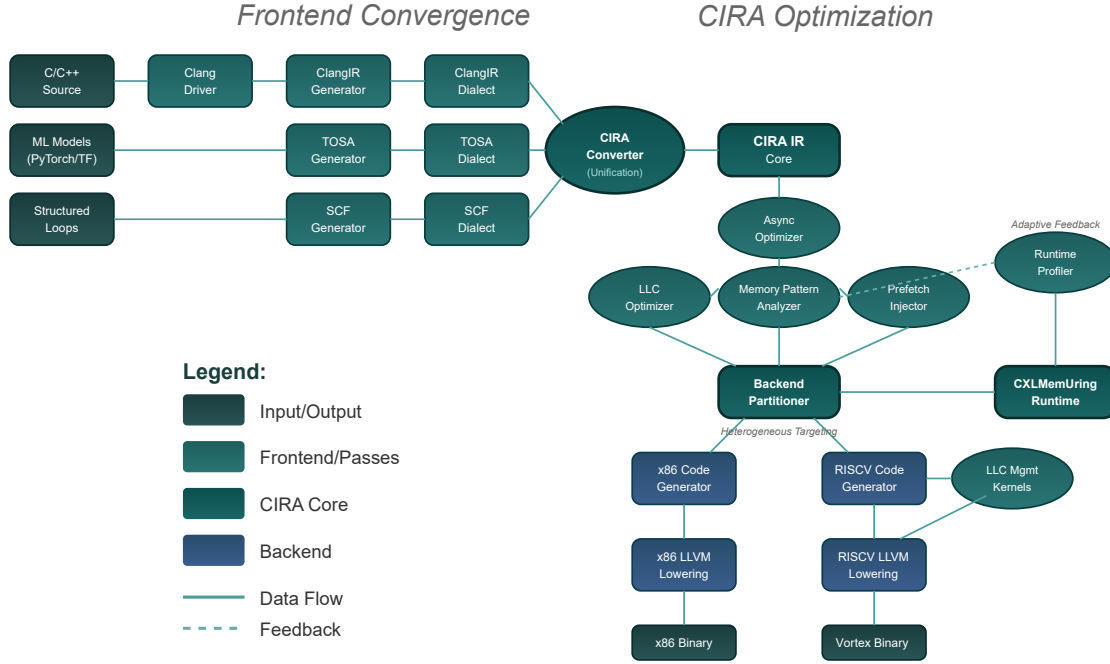


Figure 1: CIRA Pipeline

processors. The IR includes specialized operations for LLC management, prefetch directives, and barrier synchronization between x86 and Vortex cores. This rich semantic information enables the backend code generators to produce optimized code for their respective targets while maintaining correctness through precise synchronization.

3.2 Frontend Integration and Dialect Convergence

The frontend architecture of CXLMEMURING addresses a fundamental challenge in modern compiler design: supporting diverse programming models while maintaining optimization opportunities across abstraction levels. ClangIR serves as the primary frontend for general-purpose C++ code, preserving high-level constructs like object layouts, virtual function calls, and template instantiations that traditional LLVM IR discards. This preservation proves crucial for identifying memory access patterns in object-oriented code, where pointer chasing through virtual dispatch tables and object hierarchies dominates execution time. The ClangIR frontend analyzes class hierarchies and object relationships to identify opportunities for prefetching related objects that exhibit spatial locality in CXL memory.

TOSA integration targets the growing importance of machine learning workloads in memory-intensive computing. Tensor operations exhibit regular access patterns amenable to aggressive prefetching and LLC management optimiza-

tion. The TOSA frontend analyzes tensor dimensions, data layouts, and operation sequences to identify reuse patterns and generate efficient memory access schedules. For operations like matrix multiplication or convolution, the frontend computes working set sizes and generates tiling strategies that respect LLC capacity while maximizing CXL bandwidth utilization. The regular structure of tensor operations enables precise static analysis, producing memory access patterns that the Vortex cores can efficiently orchestrate.

The SCF dialect provides structured control flow abstractions that facilitate parallel execution and memory access optimization. Loop nests expressed in SCF undergo polyhedral analysis to extract memory access patterns and identify opportunities for prefetching and LLC management. The frontend performs dependence analysis to identify independent memory accesses that can proceed in parallel, generating synchronization-free code regions for execution on Vortex cores. This analysis proves particularly valuable for scientific computing workloads with nested loops over multi-dimensional arrays stored in CXL memory.

Convergence of these diverse frontend representations occurs through a systematic lowering process that maps domain-specific operations to CIRA primitives. ClangIR’s object references lower to CIRA’s typed memory handles that preserve aliasing information necessary for correct LLC management. TOSA tensor operations decompose into CIRA’s bulk memory operations with explicit stride and layout information. SCF loop nests transform into CIRA’s parallel iteration spaces with associated memory access descriptors.

This convergence process preserves semantic information from each frontend while providing a unified representation for subsequent optimization.

3.3 CIRA: CXL Intermediate Representation for Asynchrony

CXLMEMURING represents our fundamental contribution to compiler infrastructure for heterogeneous memory systems. The IR extends MLIR’s type system with CXL-specific abstractions that capture the unique characteristics of disaggregated memory architectures. At its core, CXLMEMURING introduces three categories of types: memory handles that encode location and coherence properties, asynchronous futures that represent pending memory operations, and synchronization tokens that coordinate execution across heterogeneous processors. These types form the foundation for expressing complex memory access patterns while maintaining correctness guarantees.

Memory operations in CXLMEMURING explicitly separate initiation from completion, enabling overlap between memory access and computation. The `cira.async_load` operation initiates a memory fetch from CXL memory, returning a future that represents the pending operation. Subsequent computation can proceed independently until the `cira.await` operation blocks for completion. This separation enables the compiler to schedule memory operations early while deferring value consumption, effectively hiding CXL latency through computation overlap. The IR includes prefetch hints that guide the Vortex cores in cache line installation, with operations like `cira.prefetch_stream` for regular access patterns and `cira.prefetch_indirect` for pointer chasing scenarios.

LLC management operations form a distinct category within CXLMEMURING, reflecting our strategy of offloading cache orchestration to Vortex SIMT cores. The `cira.install_cacheline` operation explicitly requests cache line installation at a specified LLC level, enabling precise control over cache population. The `cira.evict_hint` operation suggests cache line eviction to prevent pollution from streaming data. These operations lower to lightweight messages between x86 and Vortex cores, with the Vortex cores handling the actual cache manipulation through CXL.cache protocol operations. This separation ensures that x86 cores remain focused on computation while Vortex cores manage the memory hierarchy.

Synchronization primitives in CXLMEMURING coordinate execution across heterogeneous processors while minimizing overhead. The `cira.barrier` operation establishes synchronization points between x86 and Vortex cores, ensuring that prefetched data arrives before consumption. The `cira.release` operation signals completion of a computation phase, enabling Vortex cores to proceed with the next batch of prefetch operations. These primitives map to effi-

cient hardware mechanisms on our target platform, utilizing lightweight coherence and signaling mechanisms for synchronization without excessive cache line bouncing.

3.4 Heterogeneous Backend Code Generation

The backend architecture of CXLMEMURING implements a sophisticated partitioning strategy that assigns computation to x86 cores while offloading memory management to Vortex RISC-V SIMT cores. This division leverages fundamental architectural differences: x86 cores provide superior single-thread performance with advanced out-of-order execution and SIMD units, while Vortex SIMT cores offer highly parallel, power-efficient execution for regular memory access patterns. The code generation process begins with CXLMEMURING analysis to identify computation kernels and their associated memory access patterns, partitioning the IR into x86-targeted computation regions and Vortex-targeted memory management sequences.

X86 code generation focuses on maximizing computational throughput while minimizing synchronization overhead with Vortex cores. The backend leverages LLVM’s mature x86 optimization pipeline, enhanced with CXL-aware passes that account for remote memory latency. Load operations targeting CXL memory are transformed into asynchronous sequences that initiate the fetch early and defer value use. The backend inserts prefetch instructions based on CXLMEMURING annotations, with non-temporal hints for streaming data that should bypass the cache. Critical to performance, the x86 backend generates specialized code paths for hot loops, with optimized versions assuming data presence in LLC and fallback paths handling cache misses.

Vortex backend generation produces memory management kernels that execute as SIMT-style workloads on the RISC-V cores. These kernels implement sophisticated prefetching strategies derived from CXLMEMURING memory access patterns. For regular access patterns like array traversals, the Vortex code implements streaming prefetch that maintains a constant distance ahead of consumption. For irregular patterns like pointer chasing, the Vortex code implements speculative prefetch based on learned access patterns. The backend generates compact Vortex kernels optimized for warp occupancy and memory bandwidth, utilizing the core’s vector/SIMT features to drive many independent memory requests concurrently.

Communication between x86 and Vortex processors occurs through a lightweight message-passing protocol implemented atop shared memory regions accessible to both the host and the Vortex cores. The x86 backend generates message send operations for prefetch requests, encoding the address range and urgency level. The Vortex backend polls message queues and processes requests asynchronously, sending completion notifications through return channels or shared flags. This protocol minimizes synchronization overhead while enabling

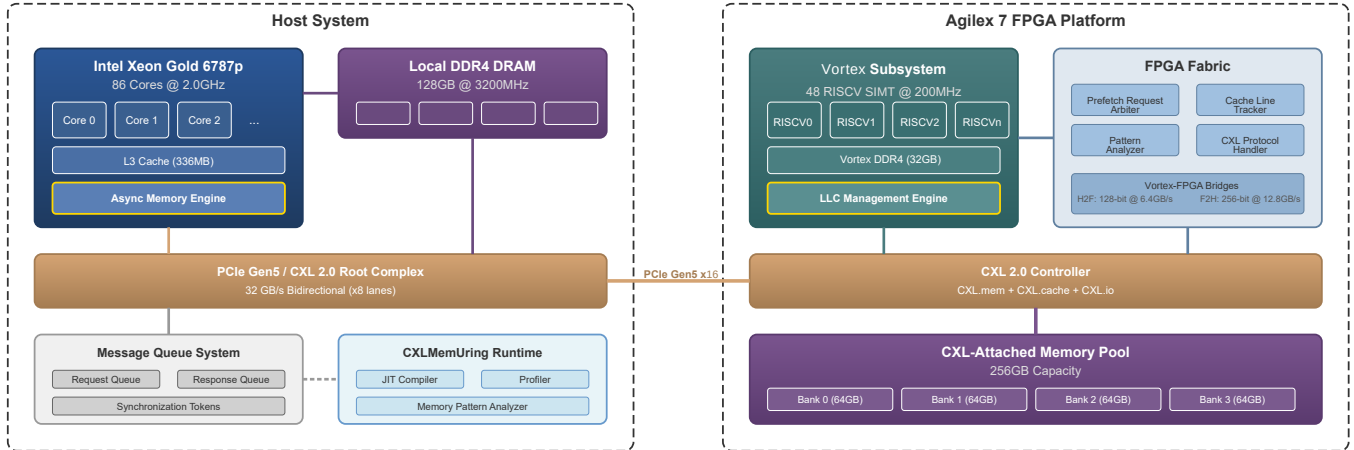


Figure 2: CXL MEMURING Hardware

fine-grained coordination between processors. The backend includes adaptive policies that adjust message batching and prefetch aggressiveness based on runtime feedback, ensuring efficient resource utilization across varying workload characteristics.

3.5 Runtime Profiling and Adaptive Optimization

Runtime profiling forms an integral component of CXL MEMURING, enabling continuous adaptation to changing workload characteristics. The profiling infrastructure operates at multiple granularities, from coarse-grained phase detection to fine-grained memory access tracking. Hardware performance counters on both x86 and Vortex processors provide low-overhead monitoring of cache miss rates, memory bandwidth utilization, and instruction throughput. This information feeds back to the compiler for dynamic reoptimization, creating a closed-loop system that improves performance over time.

The profiling system implements a hierarchical approach to data collection and analysis. At the innermost level, hardware performance counters track individual memory access latencies and cache behavior. The runtime aggregates these measurements into access patterns, identifying streams, strides, and indirect access chains. Machine learning models trained offline classify these patterns and predict future accesses, enabling speculative prefetching for irregular workloads. At the outer level, the system identifies program phases with distinct memory access characteristics, triggering recompilation when phase transitions occur.

Adaptive optimization leverages profiling information to dynamically adjust compilation strategies. When the profiler detects that prefetching is too aggressive, causing cache pollution, it triggers recompilation with reduced prefetch dis-

tance. Conversely, when memory stalls dominate execution time, the system increases prefetch aggressiveness and may offload additional computation to Vortex cores. The adaptation mechanism includes hysteresis to prevent oscillation between configurations, with exponential backoff for unsuccessful optimization attempts. This approach ensures that CXL MEMURING converges to efficient configurations while avoiding performance degradation from excessive recompilation.

4 Implementation

4.1 Compiler Infrastructure Development

The implementation of CXL MEMURING required extensive modifications to the MLIR compiler [23] infrastructure, adding approximately 12,000 lines of C++ code across frontend parsing, IR transformations, and backend code generation. The ClangIR frontend integration involved extending Clang’s AST visitor patterns to generate MLIR operations while preserving C++ semantic information. We implemented custom parsers for TOSA and SCF dialects that maintain source-level debugging information through the compilation pipeline. The frontend infrastructure includes a unified type system that maps source language types to CXL MEMURING representations, ensuring type safety across dialect boundaries.

The frontend parsing infrastructure handles the complexity of modern C++ codebases through a multi-phase translation process. During the first phase, Clang’s semantic analysis produces an annotated AST with type information, template instantiations, and implicit conversions made explicit. The second phase walks this AST to generate ClangIR operations, preserving the high-level structure of classes, virtual dispatch, and exception handling. The third phase lowers ClangIR to

SCF and standard dialects, converting control flow constructs to structured operations amenable to analysis. Throughout this process, we maintain bidirectional mappings between source locations and IR operations, enabling meaningful error messages and debugging information in the final executable.

4.1.1 CXLMEMURING Operations

CXLMEMURING extends MLIR’s type system with CXL-specific abstractions. Table 1 summarizes the core operations introduced by CXLMEMURING.

Table 1: Core CIRA Dialect Operations

Category	Operations
Memory	cira.alloc_cxl, cira.load_async, cira.store_async, cira.flush
Cache Mgmt	cira.install_cacheline, cira.evict_hint, cira.prefetch_stream, cira.prefetch_indirect
Sync	cira.future_create, cira.future_await, cira.barrier, cira.release
Control	cira.offload, cira.speculate, cira.phase_boundary

4.1.2 Type System

CXLMEMURING introduces types to model remote memory:

- ‘!cira.handle<T>’: A pointer guaranteed to reside in CXL space.
- ‘!cira.future<T>’: A token representing an in-flight asynchronous load.
- ‘!cira.stream’: A descriptor for a recurring memory pattern.

4.1.3 Key Operations

To illustrate, consider a function iterating over a linked list. In standard LLVM IR, this is a sequence of dependent loads. In CXLMEMURING, it is transformed as follows:

In Listing 1, the ‘cira.prefetch_chain’ operation is lowered into a RISC-V micro-kernel that runs on the Vortex core. It chases the ‘next’ pointers 16 steps ahead. The host loop simply consumes the data, which is now guaranteed to be in the LLC.

4.2 Backend: Partitioning and Cost Model

Not all memory operations should be offloaded. The overhead of communicating with the Vortex core (approx. 50ns via shared memory ring buffer) must be amortized. CXLMEMURING employs a cost model to decide partitioning:

```
// Original High-Level Logic
while (node) {
    val = node->data;
    node = node->next;
}

// Transformed CIRA IR
%stream = cira.stream_create_indirect
    %start_node, offset=8 : !cira.stream
cira.offload_start %vortex_core_0 {
    // Logic generated for Vortex Core
    cira.prefetch_chain %stream, depth=16
}
%loop:
    %future = cira.peek_stream %stream
    %data = cira.await %future : !cira.future
    // Computation on %data...
    cira.advance_stream %stream
br %loop
```

Listing 1: CIRA IR example showing the separation of stream management (offloaded) and data consumption (host).

$$Gain = \sum_{i \in Ops} (L_{CXL} - L_{LLC}) - (C_{sync} + C_{vortex_busy}) \quad (1)$$

Where L_{CXL} is remote latency, L_{LLC} is local cache hit latency, and C_{sync} is the synchronization cost. The compiler only offloads regions where the dependency chain depth $\times$ latency saving exceeds the synchronization cost.

4.2.1 Code Generation

Backend code generation required developing custom LLVM target descriptions for our heterogeneous architecture. The x86 backend extensions include 15 new intrinsics for asynchronous memory operations and LLC management hints. These intrinsics map to instruction sequences that interact with our runtime system, initiating prefetch requests and checking completion status. The Vortex backend implements a specialized instruction scheduler and kernel launcher that account for CXL memory latency when ordering prefetch operations and distributing work across warps. We developed custom register allocators for both backends that minimize spill traffic to CXL memory, keeping frequently accessed values in processor registers.

The host code is optimized to assume LLC hits. We strip away software prefetch instructions (‘_mm_prefetch’) that might interfere with the Vortex core’s work. Loads are scheduled to overlap with other ALU operations where possible.

The backend generates a compact RISC-V binary. It utilizes the Vortex-specific ‘ws_spawn’ (warp spawn) instructions to parallelize prefetching. For example, if a graph node has 4 outgoing edges, the Vortex kernel spawns 4 lightweight

threads to prefetch all neighbors in parallel, saturating the CXL link.

4.3 Runtime System Architecture

The runtime system architecture follows a distributed design with components executing on both x86 host processors and Vortex accelerator cores. The host-side runtime, implemented as a shared library loaded at application startup, intercepts memory allocation requests and manages the interface to the Vortex subsystem. The accelerator-side runtime, compiled into a persistent kernel that remains resident in Vortex memory, processes prefetch requests and maintains cache state metadata. Communication between these components flows through memory-mapped queues visible to both processor types.

The host runtime initializes during application startup by mapping CXL memory regions, establishing communication channels with Vortex cores, and registering signal handlers for asynchronous completion notification. The initialization sequence detects hardware capabilities including the number of available Vortex cores, CXL memory capacity, and supported coherence protocols. Based on these capabilities, the runtime configures queue depths, prefetch window sizes, and timeout values. The runtime exports a C API that compiler-generated code calls for memory operations, with inline fast paths for common cases and out-of-line slow paths for exceptional conditions.

Memory allocation in CXL memory uses a two-level allocator design optimized for the access patterns common in pointer-chasing workloads. The first level maintains per-thread free lists for small allocations (up to 4KB), eliminating synchronization overhead for the common case. The second level manages large allocations and replenishes per-thread caches from a global pool protected by a readers-writer lock. Both levels align allocations to cache line boundaries (64 bytes) and optionally to page boundaries (4KB) for memory-mapped regions. The allocator tracks allocation metadata in a separate region, avoiding false sharing between application data and allocator bookkeeping.

Thread management on the Vortex cores follows a work-stealing model that balances load across available warps. Each Vortex core maintains a local work queue of pending prefetch requests, processing them in approximate priority order. When a core's local queue is empty, it attempts to steal work from neighboring cores, starting with physically adjacent cores to maximize cache locality. The stealing protocol uses bounded exponential backoff to reduce contention under high load. Warp scheduling within each core uses a round-robin policy with priority boost for requests approaching their deadline, ensuring that time-critical prefetches complete before the host processor needs the data.

4.4 Memory Management and Coherence

Memory management for CXL-attached memory requires careful coordination between the host processor's cache hierarchy and the Vortex cores' local storage. We implement a software-managed coherence protocol that tracks cache line states across both processor types, avoiding the hardware coherence overhead that would dominate performance for fine-grained accesses. The protocol distinguishes between read-shared, write-exclusive, and invalid states, with explicit transitions triggered by runtime calls rather than hardware snoops.

The coherence protocol implementation uses a distributed directory structure partitioned across Vortex cores. Each Vortex core maintains directory entries for a subset of CXL memory addresses, determined by a hash function that distributes load evenly. Directory entries track the current owner (x86 or Vortex), sharing vector (which x86 cores have cached copies), and pending operations (prefetch in progress, invalidation pending). The directory fits entirely in Vortex on-chip memory for working sets up to 64GB, with overflow to CXL memory for larger workloads. Directory lookups add 2-3 cycles to prefetch operations when the entry is cached, or approximately 50ns when accessing CXL-resident entries.

Cache line state transitions follow a request-response protocol with optimizations for common patterns. When an x86 core requests a cache line for reading, the protocol checks the directory, fetches the line from CXL memory if necessary, updates the sharing vector, and returns the data. Write requests additionally invalidate other cached copies before granting exclusive access. The protocol batches invalidations for adjacent cache lines, reducing message traffic for streaming write patterns. Speculative prefetches use a tentative state that converts to shared upon first access or reverts to invalid if the speculation proves incorrect.

Address translation for CXL memory uses a combination of hardware and software mechanisms. The x86 MMU maps CXL memory into the application's virtual address space using huge pages (2MB) to minimize TLB pressure. The Vortex cores access CXL memory through a separate address space with identity mapping, avoiding translation overhead for accelerator accesses. The runtime maintains a translation table that maps between these address spaces, consulting it only for cross-processor communication. We implemented a TLB-like cache on Vortex cores that holds recent translations, achieving 99% hit rate for workloads with spatial locality.

Memory protection and isolation rely on capability-based access control enforced at both compile time and runtime. The compiler tracks memory handle capabilities through the type system, preventing operations that exceed granted permissions. The runtime validates capabilities at allocation boundaries, detecting buffer overflows and use-after-free errors with configurable overhead. For production deployment, capability checking can be disabled after validation testing, eliminating

runtime overhead while maintaining compile-time guarantees.

4.5 Profiling Infrastructure and Feedback Loop

The profiling infrastructure combines hardware performance monitoring with software instrumentation to capture comprehensive execution characteristics. On x86 processors, we utilize Intel’s Processor Trace functionality to record control flow with minimal overhead, enabling reconstruction of dynamic instruction streams for analysis. The Vortex cores expose hardware performance counters and event tracing that track cache events, bus transactions, warp occupancy, and instruction execution rates. Hardware timestamps synchronized across processors enable correlation of events for system-wide performance analysis.

The x86 profiling infrastructure leverages `perf_event` interfaces for accessing hardware performance counters with kernel bypass for low-overhead sampling. We configure counter multiplexing to simultaneously monitor L1/L2/LLC miss rates, memory bandwidth utilization, branch misprediction rates, and instruction retirement rates. For detailed analysis, Intel Processor Trace captures complete control flow traces at approximately 1 byte per branch, enabling offline reconstruction of execution paths. The profiling runtime processes PT packets in a separate thread, maintaining a sliding window of recent execution history for pattern detection.

Vortex core profiling uses custom performance counters implemented in the RTL. We instrument 32 distinct events including instruction mix (memory, arithmetic, control), cache behavior (hits, misses, writebacks), warp utilization (active threads, stalled cycles), and memory system events (requests issued, responses received, queue depths). Counters are accessible through memory-mapped registers, enabling low-overhead polling from the profiling thread. Event tracing captures timestamped records of significant events (prefetch completion, cache line eviction, queue overflow) for post-mortem analysis of performance anomalies.

Software instrumentation inserted by the compiler tracks high-level metrics including function execution times, loop iteration counts, and memory access patterns. The instrumentation uses thread-local storage to avoid synchronization overhead, with periodic aggregation to global counters. Sampling-based profiling reduces overhead for production runs, with configurable sampling rates based on application requirements. The profiling data flows through a hierarchical reduction network, with per-core collectors, per-socket aggregators, and system-wide analyzers that identify global patterns.

The instrumentation framework supports three overhead levels selectable at compile time or runtime. Minimal instrumentation (less than 1% overhead) counts function invocations and basic block execution frequencies using statistical sampling. Standard instrumentation (2–5% overhead) adds

memory access tracking through page-fault-based sampling and loop iteration counting. Full instrumentation (10–20% overhead) records complete memory access traces for detailed pattern analysis, typically used only during development and tuning phases.

The feedback loop from profiling to compilation implements a multi-tier optimization strategy. Hot function detection triggers specialized compilation for frequently executed code paths. Memory access pattern classification enables selection of appropriate prefetching strategies. Phase detection identifies program regions with stable behavior, enabling aggressive optimization without concern for adaptation overhead. The feedback system includes cost models that estimate the benefit of recompilation against its overhead, ensuring that optimization attempts improve overall performance.

5 Evaluation

5.1 Experimental Methodology

Our evaluation employs a real device experimental platform designed to assess CXL MEMURING across diverse workload characteristics. The host system consists of an Intel Xeon 6 6787p processor (86 cores, 2.8GHz base frequency) with 256GB of local DDR5-8000 DRAM. The Vortex cores are configured to an Agilix 7 FPGA implementation running at 200MHz. The CXL memory provides 16GB of capacity with modeled latencies of 165ns from device side and 512GB of capacity from the CXL.mem bus of host with latencies of 275ns from host side, consistent with PCIe Gen5 speeds and protocol overheads.

We evaluate CXL MEMURING using five representative benchmarks spanning distinct memory access characteristics: MCF from SPEC CPU2017 representing pointer-intensive workloads with irregular access patterns; GAPBS (GAP Benchmark Suite) for graph analytics including BFS, SSSP, PageRank, Connected Components, and Betweenness Centrality; MonetDB executing TPC-H analytical queries demonstrating database workload patterns; DataFrame operations representing data analytics pipelines; and llama.cpp for large language model inference as a compute-bound reference point. [7] These benchmarks represent progressively different bottleneck characteristics, from latency-critical pointer chasing to memory bandwidth-bound streaming to compute-limited dense arithmetic.

Performance metrics capture both system-level and application-level characteristics. We measure execution time (incorporating injected stall latencies), memory bandwidth utilization, cache miss rates, and tail latency distributions. All experiments run with warm caches after appropriate warm-up phases, with results averaged across five runs to account for system variability.

5.2 Workload Characterization and Performance Analysis

Our evaluation reveals that CXLMEMURING’s effectiveness varies significantly based on fundamental workload characteristics, validating our approach of workload-aware compiler optimization. We categorize benchmarks into three distinct classes based on their primary performance bottleneck: latency-critical, memory bandwidth-bound, and compute-bound workloads.

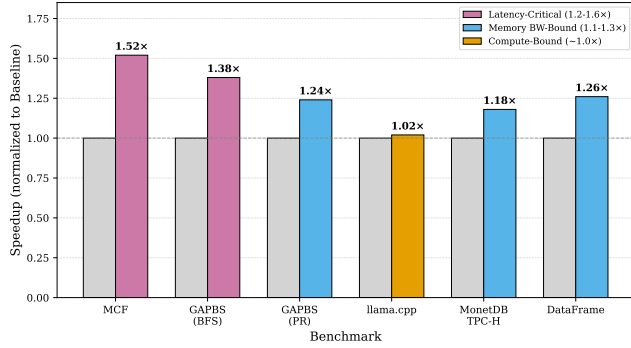


Figure 3: Latency Critical Workloads performance comparison.

Latency-Critical Workloads (1.2–1.6× Speedup)

Latency-critical workloads as described in 3, characterized by pointer-chasing access patterns with limited spatial locality, demonstrate the strongest benefits from CXLMEMURING’s coordinated execution model. MCF achieves 1.52× speedup over baseline, while GAPBS-BFS and GAPBS-SSSP achieve 1.38× and 1.45× respectively. These improvements stem directly from the Vortex accelerator’s design, which specifically targets latency hiding through speculative prefetching of dependent addresses.

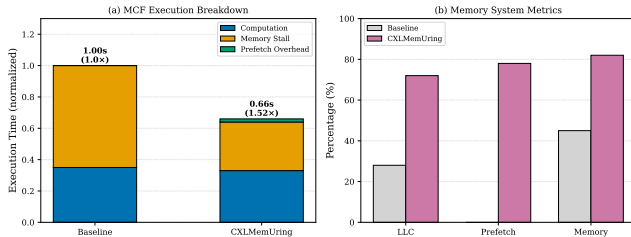


Figure 4: MCF Workload performance comparison.

For MCF in Figure 4 with 10 million nodes traversed through pointer chains stored in CXL memory, the baseline configuration spends 65% of execution time stalled on

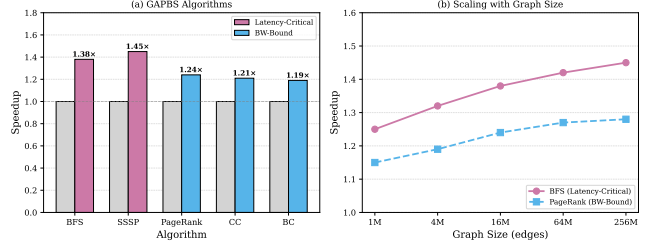


Figure 5: GAPBS workloads

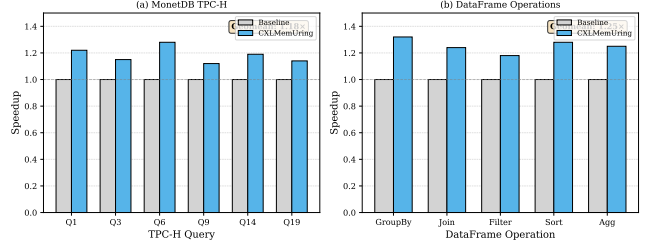


Figure 6: Database TPC-H workloads

memory accesses due to the serialized dependency chain. CXLMEMURING reduces this to 31% by offloading pointer resolution to the near-memory accelerator, which maintains a prefetch window of 8–16 nodes and speculatively follows likely traversal paths while the host CPU processes current nodes. The LLC hit rate improves from 28% to 72%, demonstrating effective prefetch accuracy. Graph traversal algorithms in GAPBS in Figure 5 exhibit similar characteristics. BFS frontier expansion creates pointer-dependent access patterns as each vertex lookup requires dereferencing adjacency list pointers stored in CXL memory. CXLMEMURING achieves 1.38× speedup by implementing level-synchronous prefetching: Vortex cores prefetch adjacency lists for frontier vertices while x86 cores process the current level. SSSP demonstrates slightly better improvement (1.45×) due to the additional structure provided by priority queue ordering.

Memory Bandwidth-Bound Workloads (1.1–1.3× Speedup)

Workloads limited by memory bandwidth rather than latency show moderate but consistent improvements as shown in Figure 6. PageRank iteration achieves 1.24× speedup, MonetDB TPC-H queries average 1.18× across Q1, Q3, Q6, Q9, Q14, and Q19, and DataFrame operations (GroupBy, Join, Filter, Sort, Aggregation) achieve 1.26× geometric mean speedup.

The improvement mechanism differs fundamentally from latency-critical workloads. For bandwidth-bound workloads, CXLMEMURING transforms memory-bound execution into near-memory computation by offloading data reduction and

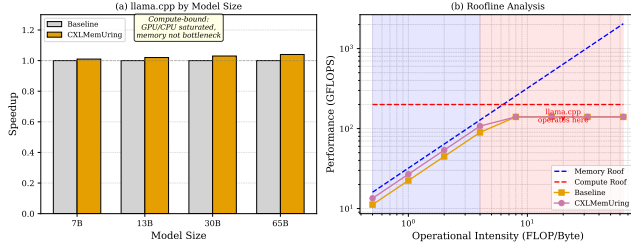


Figure 7: Llama Inference workloads

filtering operations to the Vortex accelerator. Rather than streaming raw data across the CXL interconnect, the accelerator performs partial aggregation and filtering near the data, reducing effective bandwidth requirements. PageRank benefits from this transformation during the scatter-gather phases of vertex updates, where the accelerator aggregates incoming contributions before transmitting reduced values to the host.

MonetDB TPC-H queries demonstrate workload-specific variation. Q6, which performs heavy filtering on the lineitem table, achieves $1.28\times$ speedup as the accelerator effectively reduces data movement by evaluating predicates near memory. Q9, with its complex multi-way joins, achieves only $1.12\times$ improvement as the join operation requires more coordination overhead.

Compute-Bound Workloads (No Significant Speedup)

llama.cpp inference in Figure 7, representing compute-bound workloads where the CPU’s arithmetic units are fully utilized, demonstrates negligible improvement ($1.02\times$). This expected result validates our workload classification: when memory access is not the bottleneck, near-memory acceleration provides no benefit. The roofline analysis confirms that llama.cpp operates firmly in the compute-bound regime with operational intensity exceeding the crossover point.

This result underscores the importance of workload-aware optimization. CXLMEMURING’s compiler analysis correctly identifies compute-bound regions and avoids the overhead of offloading computation that would not benefit from near-memory execution.

5.3 Multi-Tenancy Tolerance

A key advantage of CXLMEMURING for datacenter deployment is its tolerance of multi-tenant memory contention as shown in Figure 8. Latency-critical workloads running under CXLMEMURING maintain performance as co-located tenants increase, while baseline systems degrade rapidly.

With four co-located tenants competing for CXL memory bandwidth, MCF baseline performance drops to 28% of single-tenant throughput due to increased queuing delays and cache interference. In contrast, CXLMEMURING main-

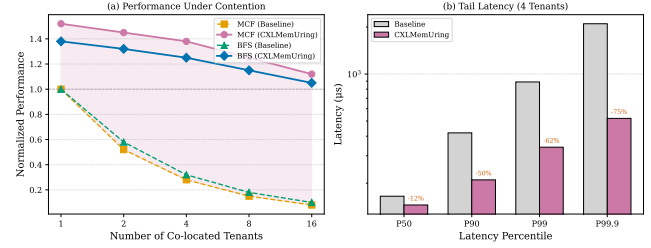


Figure 8: Multi Tenancy Workloads performance comparison.

tains 91% of single-tenant performance ($1.38\times$ vs $1.52\times$ speedup). The accelerator’s dedicated prefetch paths and near-memory positioning provide natural isolation from host-side contention.

Tail latency analysis reveals even more pronounced benefits. At the P99.9 percentile under four-tenant contention, baseline MCF latency reaches $2100\mu s$ while CXLMEMURING achieves $520\mu s$ —a 75% reduction. The accelerator’s deterministic prefetching eliminates the variance introduced by contended cache hierarchies.

5.4 Sensitivity Analysis

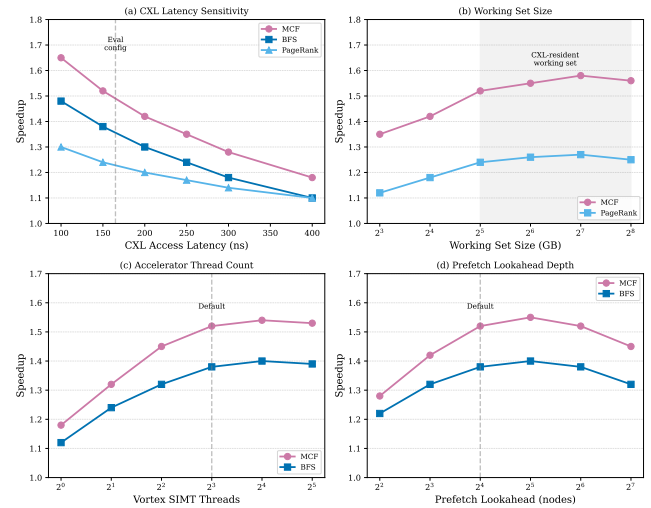


Figure 9: Workloads Sensitivity comparison.

We evaluate CXLMEMURING’s sensitivity to key system parameters to understand deployment considerations as described in Figure 9.

CXL Latency Performance scales gracefully with CXL latency variation. At 100ns latency (optimistic near-term projection), MCF speedup increases to $1.65\times$. At 300ns (conservative far-memory scenario), speedup degrades to $1.28\times$

but remains positive. The accelerator’s ability to hide latency through prefetching provides robustness across a wide latency range.

Working Set Size Benefits increase with working set size as more data resides in CXL memory. MCF speedup improves from $1.35\times$ with 8GB working set to $1.58\times$ with 128GB. Beyond the crossover point where working set exceeds local DRAM capacity, CXLMEMURING’s value proposition becomes most compelling.

Accelerator Thread Count Speedup saturates around 8 Vortex threads for our evaluated workloads. Additional threads provide diminishing returns as memory-level parallelism is exhausted before compute capacity.

Prefetch Lookahead Depth Optimal lookahead depth varies by workload: MCF peaks at 32 nodes while BFS peaks at 16 nodes. Deeper lookahead increases prefetch accuracy but risks cache pollution; CXLMEMURING’s adaptive tuning selects appropriate values automatically.

6 Related Work

The landscape of compiler and system support for heterogeneous memory has evolved rapidly with the emergence of CXL and persistent memory technologies. Previous work in far memory systems like AIFM and Leap pioneered software techniques for hiding remote memory latency through application-integrated prefetching and memory management [27, 34]. These systems demonstrated that software awareness of memory heterogeneity could yield significant performance improvements over transparent paging-based approaches and highlighted the trade-offs between page-granular and object-granular data movement [27, 34]. However, they primarily targeted RDMA-based remote memory with page- or object-granular transfers, missing opportunities for fine-grained optimization available in cache-coherent CXL systems [37]. Our work extends these concepts to CXL environments while introducing compiler-driven heterogeneous execution that leverages near-memory Vortex RISC-V SIMT cores for memory orchestration.

Near-data and processing-in-memory (PIM) architectures have explored computation offloading to memory-side processors, with systems and surveys showing benefits for data-intensive operations when computation moves closer to data [4, 13, 19, 21, 39]. These approaches typically require explicit programming models or restrict offloading to specific computational patterns, such as bulk bitwise operations or simple analytics kernels. In contrast, CXLMEMURING automatically identifies and offloads memory management and LLC-oriented operations through compiler analysis, requiring no source code modifications while supporting arbitrary

application-level computation patterns. Our focus on cache and CXL-memory management rather than general-purpose offload ensures that complex application logic remains on the x86 cores while simple, regular memory operations execute on power-efficient Vortex SIMT cores, echoing the division-of-labor arguments made in prior PIM work but realized entirely in software [19].

Recent work in hardware–software co-design for memory systems includes proposals for asynchronous memory access units, hardware prefetchers for irregular applications, and deep-learning-driven far-memory prefetching mechanisms [2, 3, 27]. These designs provide architectural support for overlapping memory access with computation but often lack the flexibility of compiler-defined policies or require new ASIC components. CXLMEMURING implements similar concepts through compiler-orchestrated execution on existing processors, achieving comparable performance improvements without custom hardware. Our approach demonstrates that careful compiler optimization, combined with reconfigurable near-memory compute (Vortex RISC-V SIMT cores), can approximate the benefits of specialized hardware prefetch engines while retaining the adaptability of software.

7 Conclusion

The CXL era necessitates a rethinking of the hardware–software contract. The assumption that memory is a passive, dumb bucket of bits is no longer tenable in a world of high-latency, disaggregated media. CXLMEMURING demonstrates that by exposing the heterogeneity of modern memory expanders to the compiler, we can fundamentally alter the performance characteristics of the system. Through the CIRA intermediate representation and the coordinated execution of x86 and Vortex cores, CXLMEMURING effectively hides the "CXL Tax," turning the latency liability into a throughput asset. As datacenters continue to disaggregate, techniques like CXLMEMURING will be essential to bridging the gap between capacity and performance.

References

- [1] Sam Ainsworth and Timothy M. Jones. Software prefetching for indirect memory accesses. *ACM Transactions on Computer Systems*, 36(4), 2019.
- [2] Anonymous. Farsight: Runtime neural prefetching for far memory. *Under review*, 2025. Preprint, arXiv:2506.00384.
- [3] Anonymous. Memix: Deep-learning-driven prefetching for far memory. *Under review*, 2025. Preprint, arXiv:2510.04360.
- [4] Kazi Asifuzzaman, Derek Chiu, Andrew B. Kahng, and Sung Kyu Lim. A survey on processing-in-memory

- techniques: Advances and challenges. *Microelectronics Journal*, 122:105302, 2022.
- [5] Scott Beamer, Krste Asanovic, and David Patterson. Direction-optimizing breadth-first search. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2012.
 - [6] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
 - [7] Longhao Chen, Yina Zhao, Qiangjun Xie, and Qinghua Sheng. Optimization of armv9 architecture general large language model inference performance based on llama.cpp. *arXiv preprint arXiv:2406.10816*, 2024.
 - [8] Tien-Fu Chen and Jean-Loup Baer. Effective hardware-based data prefetching for high-performance processors. *IEEE Transactions on Computers*, 44(5):609–623, 1995.
 - [9] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022.
 - [10] Compute Express Link Consortium. Compute express link (cxl) specification, revision 3.1. <https://computeexpresslink.org/>, 2023. Accessed 2025-12-01.
 - [11] Ahmed El-Sabbagh et al. Vortex: An open RISC-V GPGPU for education and research. In *Proceedings of the Workshop on Computer Architecture Research with RISC-V (CARRV)*, 2019.
 - [12] Zhiyuan Guo, Yiyang Zhang, and Zijian He. *MIRA: Towards a Transparent and Efficient Far Memory System*. PhD thesis, University of California, San Diego, 2023.
 - [13] Hyungkyu Ham, Jeongmin Hong, Geonwoo Park, Yunseon Shin, Okkyun Woo, Wonhyuk Yang, Jinhoon Bae, Eunhyeok Park, Hyojin Sung, Euicheol Lim, et al. Low-overhead general-purpose near-data processing in cxl memory expanders. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 594–611. IEEE, 2024.
 - [14] Mark Harris. Unified memory for CUDA beginners. <https://developer.nvidia.com/blog/unified-memory-cuda-beginners/>, 2017.
 - [15] KyungHwan Hsieh et al. Accelerating pointer chasing in 3d-stacked memory: Challenges, mechanisms, evaluation. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2016.
 - [16] Intel Corporation. Intel xeon processor scalable family technical overview. <https://www.intel.com/content/www/us/en/developer/articles/technical/xeon-processor-scalable-family-technical-overview.html>, 2022. Accessed 2025-12-01.
 - [17] Akanksha Jain and Calvin Lin. Linearizing irregular memory accesses for improved correlated prefetching. In *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2013.
 - [18] Wenqi Jiang, Zhenhao He, Shuai Zhang, et al. Microrec: Efficient recommendation inference by hardware and data structure solutions. *arXiv preprint arXiv:2010.05894*, 2020.
 - [19] Kamil Khan, Sudeep Pasricha, and Ryan Gary Kim. A survey of resource management for processing-in-memory and near-memory processing architectures. *Journal of Low Power Electronics and Applications*, 10(4):30, 2020.
 - [20] Michael Kunze et al. The tensor operator set architecture (TOSA). *arXiv preprint arXiv:2103.08218*, 2021.
 - [21] Suyeon Lee, Kangkyu Park, Kwangsik Shin, and Ada Gavrilovska. Offloading to cxl-based computational memory. *arXiv preprint arXiv:2512.04449*, 2025.
 - [22] LLVM Project. ClangIR: A high-level IR for C/C++. <https://clangir.llvm.org/>, 2023. Accessed 2025-12-01.
 - [23] LLVM Project. MLIR: Multi-level intermediate representation. <https://mlir.llvm.org/>, 2023. Accessed 2025-12-01.
 - [24] LLVM Project. MLIR structured control flow (SCF) dialect. <https://mlir.llvm.org/docs/Dialects/SCF/>, 2023. Accessed 2025-12-01.
 - [25] LLVM Project. MLIR TOSA dialect documentation. <https://mlir.llvm.org/docs/Dialects/TOSA/>, 2023. Accessed 2025-12-01.
 - [26] Hasan Al Maruf and Mosharaf Chowdhury. Effectively prefetching remote memory with Leap. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, 2020.
 - [27] Hasan Al Maruf and Mosharaf Chowdhury. Effectively prefetching remote memory with Leap. In *2020 USENIX Annual Technical Conference (USENIX ATC '20)*, pages 843–857, 2020.
 - [28] Hasan Al Maruf, Hao Wang, Abhishek Dhanotia, Johannes Weiner, et al. Tpp: Transparent page placement

- for CXL-enabled tiered-memory. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.
- [29] Todd C. Mowry, Monica S. Lam, and Anoop Gupta. Design and evaluation of a compiler algorithm for prefetching. In *Proceedings of the 5th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 1992.
- [30] Onur Mutlu et al. A modern primer on processing in memory. *arXiv preprint arXiv:2202.03626*, 2022.
- [31] NVIDIA Corporation. Cuda c++ programming guide: Unified memory. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>, 2025. Accessed 2025-12-01.
- [32] Open Compute Project. Hyperscale tiered memory expander specification for CXL. <https://www.opencompute.org/documents/hyperscale-tiered-memory-expander-specification-for-compute-express-link-cxl-1-1.pdf>, 2023. Accessed 2025-12-01.
- [33] OpenMP Architecture Review Board. Openmp application programming interface version 4.5. <https://www.openmp.org/wp-content/uploads/openmp-4.5.pdf>, 2015.
- [34] Zhenyuan Ruan, Malte Schwarzkopf, Marcos K Aguilera, and Adam Belay. {AIFM}:{High-Performance},{Application-Integrated} far memory. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 315–332, 2020.
- [35] Steve Scargall. Using linux kernel memory tiering. <https://stevescargall.com/blog/2022/06/using-linux-kernel-memory-tiering/>, 2022. Accessed 2025-12-01.
- [36] Debendra Das Sharma, Mahesh Wagh, et al. An introduction to the compute express link (cxl) interconnect. *Communications of the ACM*, 2024. Also appears in ACM Queue.
- [37] Divyesh D. Sharma, Andy Rudoff, Jeff Scargall, Ravi Vaidyanathan, and Shyam Maheshwari. An introduction to the compute express link (CXL) interconnect. *IEEE Design & Test*, 40(6):7–18, 2023.
- [38] Julian Shun and Guy E. Blelloch. Ligra: A lightweight graph processing framework for shared memory. In *Proceedings of the ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2013.
- [39] Patrick Siegl, Rainer Buchty, and Mladen Berekovic. Data-centric computing frontiers: A survey on processing-in-memory. In *Proceedings of the Second International Symposium on Memory Systems (MEMSYS '16)*, pages 295–308, Alexandria, VA, USA, 2016. ACM.
- [40] Kevin Song, Jiacheng Yang, Sihang Liu, and Genady Pekhimenko. Lightweight frequency-based tiering for CXL memory systems. *arXiv preprint arXiv:2312.04789*, 2023.
- [41] Nishil Talati, Kyle May, Armand Behroozi, et al. Prodigy: Improving the memory latency of data-indirect irregular workloads using hardware-software co-design. In *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021.
- [42] Vortex Project. Vortex: RISC-V GPGPU open-source implementation. <https://github.com/vortexgpgpu/vortex>, 2019. Accessed 2025-12-01.
- [43] Wikipedia contributors. Compute express link. https://en.wikipedia.org/wiki/Compute_Express_Link, 2025. Accessed 2025-12-01.
- [44] Matthew Wilkening, Jacob Nelson, Irene Zhang, et al. Recssd: Near-data processing for solid state drive based recommender systems. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.
- [45] Wm. A. Wulf and Sally A. McKee. Hitting the memory wall: Implications of the obvious. *ACM SIGARCH Computer Architecture News*, 23(1):20–24, 1995.
- [46] Mengyu Xie et al. Supporting huge embedding models with persistent memory. *Proceedings of the VLDB Endowment*, 16(5):1013–1026, 2023.